


Advanced Array Methods:-

① for each :- it's a method.

→ it can't be called a loop technically.
But loop bol bhi doge, tab bhi di'kkat nahi hai.

→ for each sirf array par lagta hai.

→ it accepts call back func.

→ "for each" array ke each item par operation perform karega.

```
let worldCup = ['india', 'australia', 'pakistan', 'New Zealand', 'sri lanka'];  
worldCup.forEach(function() {  
});
```

→ cb function

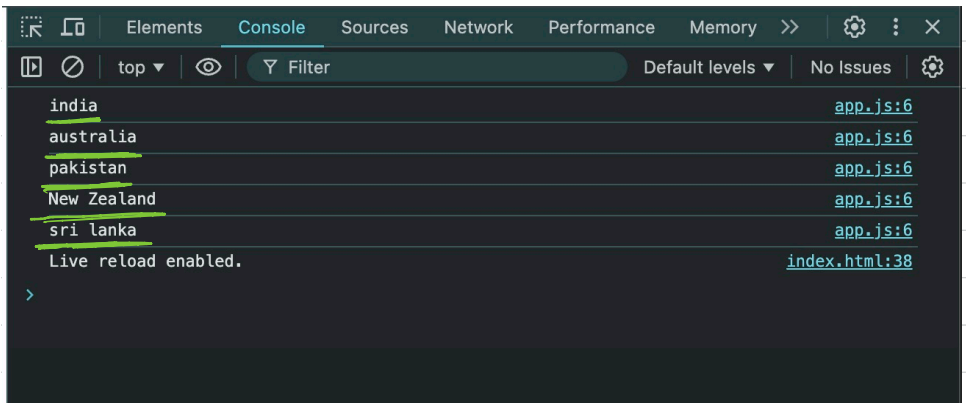
callback

for each worldCup array ke har item par function wala action perform karega.

```
worldCup.forEach(function() {  
})
```

→ cb func

```
let worldCup = ['india' , 'australia' , 'pakistan' ,  
  'New Zealand' , 'sri lanka'];  
  
worldCup.forEach( function(item) {  
  console.log(item);  
} );
```



```
worldCup.forEach ( function (item) {  
  console log (item);  
} )
```

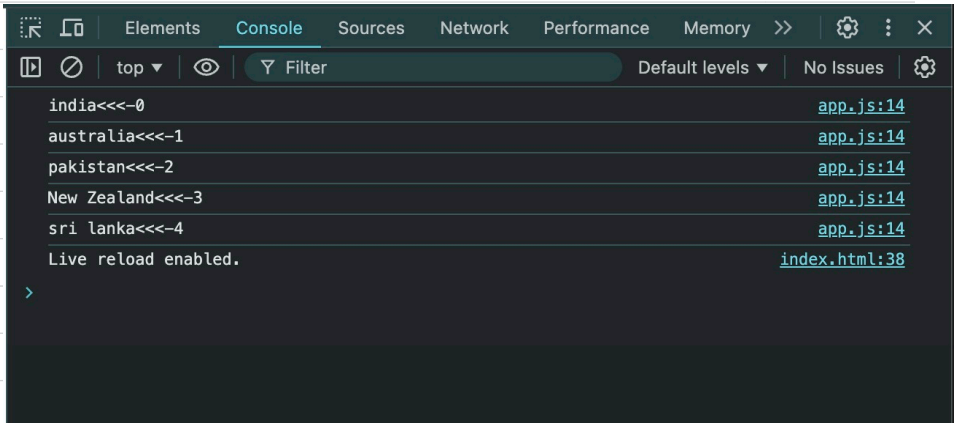
Kya ander wala function cb hai?

Ans yes , it is cb . But vo to call nahi ho saka.

Logic: Aap forEach() ke ander jo bhi
function pass karoge use
forEach call karega hi karega
but vo hamre code wai nahi
dikhega.

```
let worldCup = ['india' , 'australia' , 'pakistan' , 'New Zealand' , 'sri lanka'];
```

```
worldCup.forEach( function(item, index) {  
    console.log(item + "<<<-" + index);  
} );
```



```
india<<<-0 app.js:14  
australia<<<-1 app.js:14  
pakistan<<<-2 app.js:14  
New Zealand<<<-3 app.js:14  
sri lanka<<<-4 app.js:14  
Live reload enabled. index.html:38
```

item acting as worldCup [i]
index " " index.

forEach() ke cb fnc mei first argument arr[i] hoga
and second argument index hoga
array ka.

item → SRK
index → Salman } → variables ke name
change kar doge
tab bhi same o/p
hoga.

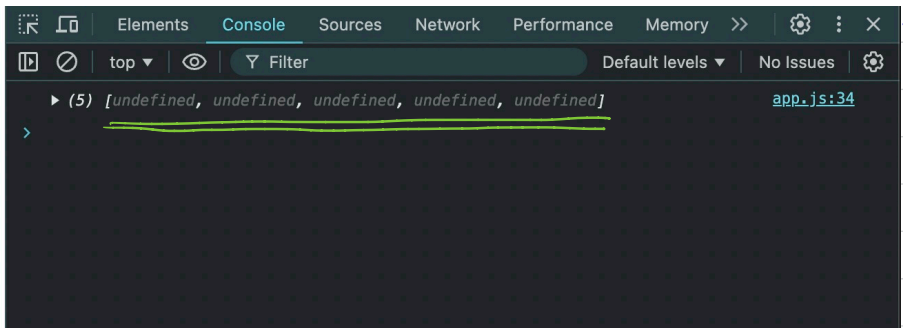
```
let worldCup = ['india' , 'australia' , 'pakistan' ,  
'New Zealand' , 'sri lanka'];
```

```
worldCup.forEach( function(srk, salman) {  
    console.log(srk + "<<<-" + salman);  
} );
```


- ② `map()` :- → it accepts a cb function.
→ only arrays par kaam karega
→ array ke har element par jayega and kaam karega.
→ ye new array return karega.
`for each()` → ye new array return nahi karta.

```
let worldCup = ['india', 'australia', 'pakistan', 'New Zealand', 'sri lanka'];  
  
let newArray = worldCup.map( function(item) {  
    return item.toUpperCase(); → ll  
} );  
  
console.log(newArray);
```

• `map()` → ek ^{new} array return karega. Agar mai `ll` na likhu, then this will be the output



agar mai `ll` likhunga tab ye output hoga

```
▶ (5) ['INDIA', 'AUSTRALIA', 'PAKISTAN', 'NEW ZEALAND', 'SRI LANKA'] app.js:34
```

Hamare code mei .map() ne array return
Kiya jisme sabhi letters Upper Case
the.

item.toUpperCase()
↪ acting as arr[i]

```
let numbers = [ 10, 20, 30, 40, 50];  
  
let newSqrArray = numbers.map(function(item , index){  
    return item*item;  
})  
  
console.log(numbers);  
console.log(newSqrArray);
```

o/p ↓

```
▶ (5) [10, 20, 30, 40, 50] app.js:44  
▶ (5) [100, 400, 900, 1600, 2500] app.js:45
```

• map() mei bhi aap .forEach() ke jaise 2 items
bhej sakte ho → jaise .map(function(item, index)
{ y }

③ . filter() :-

- it accepts a cb function.
- only arrays par kaam karega
- array ke har element par jayega and kaam karega.
- ye new array return karega.

- map() ke case mei new array and old array mei no. of elements are the same.
- filter() ke case mei no. of elements can vary b/w 0 to ^{full} length of old array
- filter() is mainly used to filter elements of array.

```
let numbers = [10, 20, 30, 40, 50];  
  
let filteredArray = numbers.filter(function(item){  
  if(item >= 30){  
    return true  
  } else {  
    return false  
  }  
});  
  
console.log(numbers);  
console.log(filteredArray);
```

↳ jaha jaha ^{hoga} true, sirf wohi wali values new array mei jayegi.

A screenshot of the Chrome DevTools Console. The 'Console' tab is active. It shows two log entries:
1. `(5) [10, 20, 30, 40, 50]` from `app.js:59`. The array `[10, 20, 30, 40, 50]` is underlined in green.
2. `(3) [30, 40, 50]` from `app.js:60`. The array `[30, 40, 50]` is underlined in green.
A green arrow points from the first log entry to the second, indicating the state of the array after the first iteration of a `reduce` operation.

④ `Reduce()` :- (Home work de diya sir ne)

```
let numbers = [1, 2, 3, 4, 5];  
  
let sum = numbers.reduce(function(item, index) {  
  console.log(`${item} + " " + ${index}`);  
  return item + index;  
}, 10);  
console.log(sum);
```

initial value

A screenshot of the Chrome DevTools Console showing the execution of the code from the previous block. The console displays a series of log entries:
10 + " " + 1
11 + " " + 2
13 + " " + 3
16 + " " + 4
20 + " " + 5
25
The first five lines correspond to the `console.log` statements inside the `reduce` callback function, showing the accumulation of the sum at each step. The final line, `25`, is the result of `console.log(sum)`. The file path `app.js` and line numbers (67, 71) are shown on the right for each log entry.

The `.reduce()` method in JavaScript is used to reduce an array to a single value by applying a callback function to each element of the array, accumulating a result as it goes.

iteration 1 : $10 + 1 \rightarrow$ first item of array
 $\hookrightarrow$ ini

iteration 2 : $11 + 2 \rightarrow$ second item of array
 $\downarrow$
result of last operation

array.reduce(callbackFunction, initialValue)

How .reduce() Works

1. `.reduce()` starts with the `initialValue` (if provided) or the first element of the array.
2. It applies the `callbackFunction` to each element of the array.
3. The `accumulator` is updated with the returned value of the `callbackFunction` after each iteration.
4. The final value of the `accumulator` is returned after the last iteration.

javascript

Copy code

```
let numbers = [1, 2, 3, 4, 5];

let sum = numbers.reduce(function(accumulator, currentValue) {
  return accumulator + currentValue;
}, 0);

console.log(sum); // Output: 15
```

Explanation:

1. Start with `accumulator = 0` (the `initialValue`).
2. Iteration 1: `accumulator = 0 + 1 → 1`
3. Iteration 2: `accumulator = 1 + 2 → 3`
4. Iteration 3: `accumulator = 3 + 3 → 6`
5. Iteration 4: `accumulator = 6 + 4 → 10`
6. Iteration 5: `accumulator = 10 + 5 → 15`

Hamne initialValue=0 di huyi hai $\Rightarrow$ isliye $0 + 1$

iteration 2:

1 + 2
↓
current array item
value from last operation

initial value
current array item

(1:17:00) Break over

```
function myFunc(){  
  let username = "goli beta";  
  
  function masti(){  
    console.log(username + ' masti nhi')  
  }  
  masti();  
}  
  
myFunc();
```

→ P1

↳ o/p ⇒ goli beta masti nhi

```
function myFunc(){  
  let username = "goli beta";  
  
  function masti(){  
    console.log(username + ' masti nhi')  
  }  
  
  return masti;  
}  
  
let returnedFunc = myFunc();  
returnedFunc();
```

⇒ o/p :
goli beta
mastī nhi

→ P2

Jab aap P1 and P2 ka hEC banaoge, Then you will understand ki closure kya hai !!

P1 Ka GEC:

→ GEC

MCP	CEP				
myFunc() definition	→ EC of myFunc() <table> <tr> <th>MCP</th><th>CEP</th></tr> <tr> <td> masti func defn username = undefined </td><td></td></tr> </table>	MCP	CEP	masti func defn username = undefined	
MCP	CEP				
masti func defn username = undefined					

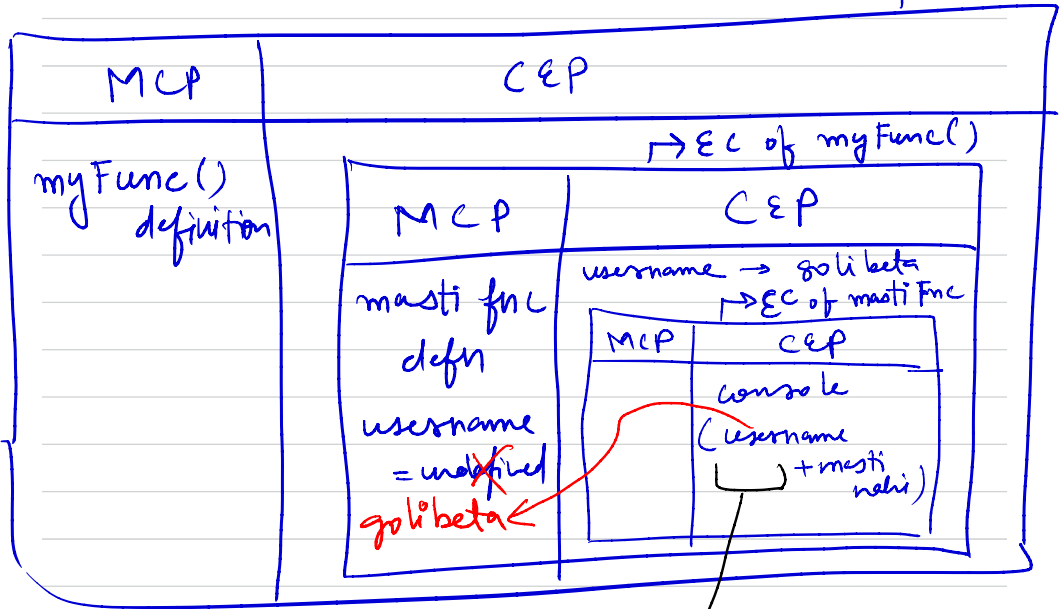


→ GEC

MCP	CEP				
myFunc() definition	→ EC of myFunc() <table> <tr> <th>MCP</th><th>CEP</th></tr> <tr> <td> masti func defn username = undefined golibeta </td><td> username updated to golibeta masti func call huq </td></tr> </table>	MCP	CEP	masti func defn username = undefined golibeta	username updated to golibeta masti func call huq
MCP	CEP				
masti func defn username = undefined golibeta	username updated to golibeta masti func call huq				



→ GEC



yaha username ki nahi
mastifnc ke MCP se
nahi aayi balki myfnc
ke MCP se aayi hai.

Aage ke steps maine draw nahi kiye. But aage
code end ho jayega. ~~go~~ one by one sabhi
Execution content of each function will be
destroyed and call stack will be empty.

GEC of P2 :-

→ GEC

MCP	C&P				
myFunc() definition returned Func = undefined myFunc()	→ EC of myFunc() <table><tr><th>MCP</th><th>C&P</th></tr><tr><td>masti fnc defn username = undefined</td><td></td></tr></table>	MCP	C&P	masti fnc defn username = undefined	
MCP	C&P				
masti fnc defn username = undefined					

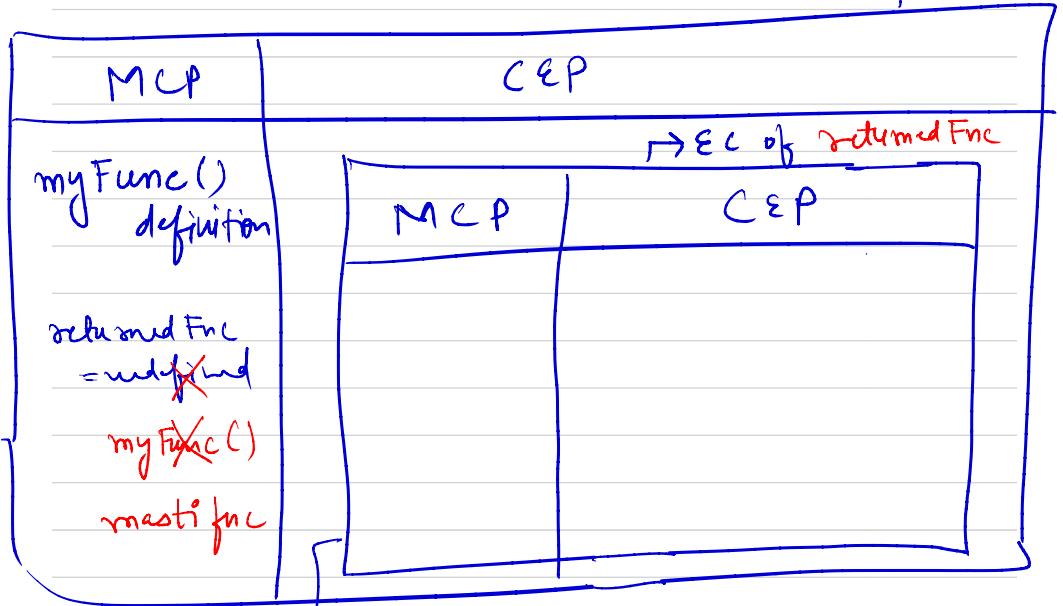


→ GEC

MCP	C&P				
myFunc() definition returned Func = undefined myFunc() masti fnc	→ EC of myFunc() <table><tr><th>MCP</th><th>C&P</th></tr><tr><td>masti fnc defn username = undefined golibeta</td><td>username → golibeta return masti</td></tr></table>	MCP	C&P	masti fnc defn username = undefined golibeta	username → golibeta return masti
MCP	C&P				
masti fnc defn username = undefined golibeta	username → golibeta return masti				



→ GEC



ye EC hai returned Func ka. mujhe console karana hai ⇒ username + "mastinahi"

But "username" aayega kaha se?

Kyunki current and parent dono ke MCP mei username variable nahi hai. Ideally to error aa jana chahiye But fir bhi alp aayega

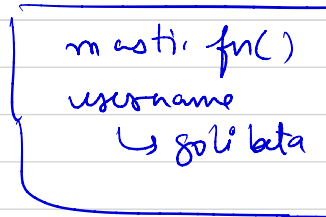
"goli beta masti nahi"

Iss concept ko closure khte hai.

whenever we return a function, that function is never returned alone. It is always returned with lexical environment along with it.

lexical environment of masti = Parent ka LE + Local memory

MCP →



↓
masti
function
ki local
memory
khaali
hai kyunki

usme koi
variable
declared
nahi hai.

(1:44:00)

p2 ka breakpoint lagake call stack samjho
plus let, var, const ka scope
detail mei
pado.

